PYSPARK COMPLETE NOTES

PHASE 1

Reading Data

Load JSON and CSV files, and take control of schemas with DDL strings and StructType.

YOU WILL

- ✓ Write spark.read.format().option().load() from memory
- ✓ Read JSON and CSV files and know which options matter
- ✓ Print and understand a schema with printSchema()
- ✓ Define a schema both as a DDL string and as a StructType
- ✓ Spot the problems in the notebook's StructType cells
- ✓ Tell an explicit schema from schema validation, and write a check for drift

START HERE IF

Phase 0 (what a DataFrame is, what lazy evaluation means).

Built from a Databricks PySpark tutorial notebook and its course outline.
Clickable contents, links and bookmarks work in any PDF viewer.

Phase 1 · Reading Data

Load JSON and CSV files, and take control of schemas with DDL strings and StructType.

YOU WILL BE ABLE TO

- ✓ Write spark.read.format().option().load() from memory
- ✓ Read JSON and CSV files and know which options matter
- ✓ Print and understand a schema with printSchema()
- ✓ Define a schema both as a DDL string and as a StructType
- ✓ Spot the problems in the notebook's StructType cells
- ✓ Tell an explicit schema from schema validation, and write a check for drift

START HERE IF

Phase 0 (what a DataFrame is, what lazy evaluation means).

IN THIS PDF

Explanations, diagrams and code from the notebook · sections tagged **Beyond the notebook** · a decoder for every term · interview questions · a data-engineering lens · a self-test with answers · a glossary

How to read the boxes

IN PLAIN WORDS
A jargon-free explanation of the idea.

DECODE THE CODE
Meaning of every function, option and symbol in the code above (first time it appears).

TIP
A habit that makes your code cleaner, faster or safer.

WATCH OUT
A trap that gives a wrong or surprising result.

NOTEBOOK OBSERVATION
Something specific noticed in the course notebook.

DATA ENGINEERING LENS
How the skill is used in real pipelines (summary page at the end).

EXTRA
Extra knowledge that is not in the notebook.

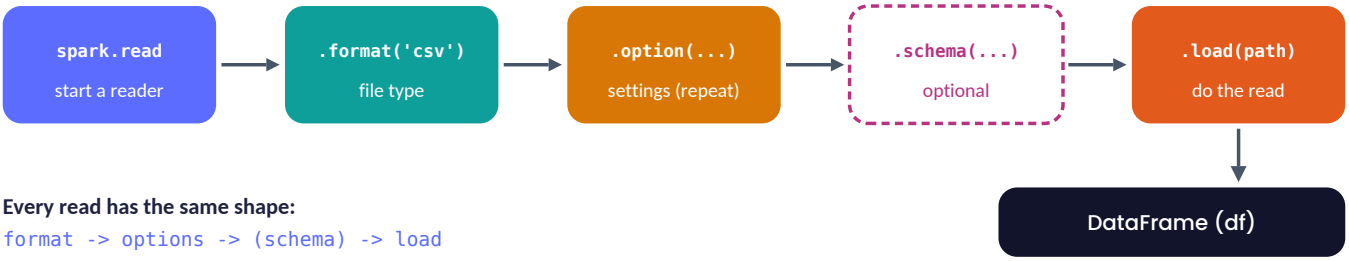
BEYOND THE NOTEBOOK
A tag under a heading: new material added to the notebook's topics, ending with an Interview corner.

CONTENTS (click an entry to jump to it)

1.1 The Spark Reader API . . . 3
1.2 Reading JSON . . . 4
1.3 Data reading utils: dbutils.fs.ls . . . 5
1.4 Reading CSV . . . 5
1.5 Schema definition and printSchema() . . . 6
1.6 DDL schema . . . 7
1.7 StructType schema . . . 8
1.8 Explicit schemas vs schema validation . . . 10
From notebook to pipeline . . . 15
Glossary . . . 15
Check yourself . . . 18
Answers and solutions . . . 19

1.1 The Spark Reader API

Every read in Spark follows one pattern: **tell Spark the format, add options, optionally give a schema, then load the path.** Learn this shape once and you can read any file type.



Every read has the same shape:
`format -> options -> (schema) -> load`

Learn this pattern once - it works for csv, json, parquet and more.

PYSPARK · GENERAL SHAPE (SKELETON)

```
df = (spark.read
        .format('csv')              # 1. the file format: csv, json, parquet, ...
        .option('header', True)     # 2. settings, one .option(key, value) at a time
        .schema(my_schema)          # 3. (optional) your own schema
        .load('/path/to/file'))     # 4. where the data is
```

DECODE THE CODE

`( ... )`	Wrapping a chain in parentheses lets it run over several lines without backslashes.
`spark.read`	Starts a **DataFrameReader**: the object you configure and then use to load a file.
`.format('csv')`	Chooses the file format: **csv**, **json**, **parquet** ... (the same call is used when writing).
`.option(key, value)`	Sets **one** setting for the reader or writer. Call it again for each extra setting.
`header`	**True** = the first row of the CSV holds column names, not data.
`.schema(my_schema)`	Hands Spark the schema to use instead of guessing it.
`.load(path)`	Points Spark at the file or folder and returns a DataFrame. (Lazy until an action needs the data.)

Piece	What it does
`spark.read`	Starts a DataFrameReader.
`.format('csv')`	Chooses the file type.
`.option(key, value)`	Reader settings. Repeat the call for each setting.
`.schema(...)`	Supplies a schema instead of letting Spark guess.
`.load(path)`	Does the read and returns a DataFrame.

THE BACKSLASH IN THE NOTEBOOK

The notebook breaks long chains across lines with a backslash `\` at the end of each line. Wrapping the whole chain in parentheses, as in the skeleton above, does the same job and avoids the classic mistake of leaving a space after the backslash.

1.2 Reading JSON

MULTILINE = FALSE (THE NOTEBOOK)

```
{"id": 1, "name": "..."}   1 record
{"id": 2, "name": "..."}   1 record
{"id": 3, "name": "..."}   1 record
```

one complete record per LINE

MULTILINE = TRUE

```
{
   "id": 1,
   "name": "...",
   "team": "..."
}
```

1 record

a record spread over several lines

The option tells Spark how the JSON is laid out in the file.

PYSPARK

```
df_json = spark.read.format('json').option('inferSchema',True)\
                    .option('header',True)\
                    .option('multiLine',False)\
                    .load('/FileStore/tables/drivers.json')
```

DECODE THE CODE

`spark`	The SparkSession - your entry point to Spark. Databricks creates it for you, already named **spark**.
`inferSchema`	**True** = Spark scans the data to guess each column's type. Without it, every CSV column is a string.
`\ (backslash)`	Line continuation: the statement carries on in the next line. Nothing may follow it on the same line.
`multiLine`	**False** = one JSON record per line. **True** = a record may span several lines.
`/FileStore/tables/`	The Databricks folder (DBFS) where uploaded files live.

PYSPARK

```
df_json.display()
```

DECODE THE CODE

`.display()`	Databricks' table viewer. It is an **action**, so it makes Spark run the job.

Part	Meaning
`format('json')`	Read the file as JSON.
`option('multiLine', False)`	Each **line** of the file is one complete JSON record (the "JSON Lines" layout). Set it to **True** when a single record or array is spread over several lines (a pretty-printed file).
`load('/FileStore/tables/drivers.json')`	The uploaded file in DBFS.
`df_json.display()`	An action: shows the DataFrame as a table.

NOTEBOOK OBSERVATION

header and **inferSchema** are CSV settings. The JSON reader works out the column names and types on its own, so those two options have no effect here. They do no harm - the tutor simply reuses the same option pattern as for CSV.

READING A PRETTY-PRINTED JSON FILE?

If the load fails or returns a single corrupt-record column, the file is probably multi-line JSON. Change to `option('multiLine', True)`.

1.3 Data reading utils: dbutils.fs.ls

PYSPARK

```
dbutils.fs.ls('/FileStore/tables/')
```

DECODE THE CODE

`dbutils.fs.ls(path)`	Databricks utility that **lists** the files in a storage folder, like *ls* in a terminal.

`dbutils` is a Databricks helper object. `dbutils.fs.ls(folder)` lists the files in a storage folder - the same thing `ls` does in a terminal. Each entry shows the full path, name, size and modification time. Use it to confirm a file was uploaded and to copy its exact path before reading it.

1.4 Reading CSV

PYSPARK

```
df = spark.read.format('csv').option('inferSchema',True).option('header',True)\
                  .load('/FileStore/tables/BigMart_Sales.csv')
```

PYSPARK

```
df.display()
```

The notebook's one-liner is shown above with a line break for readability; the code is the same.

Option	What it controls	If you leave it out
`header=True`	Treat the first row of the file as column names.	Columns are named `_c0, _c1, _c2 ...` and the header row becomes data.
`inferSchema=True`	Let Spark scan the data and guess each column's type (number, text ...).	Every column is read as **string**.

WHAT IS A SCHEMA?

A schema is the list of column names and their data types. A DataFrame always has one. The next sections show three ways to get it: let Spark infer it (this section), write it as a DDL string, or build it with StructType.

THE CSV FILE (TEXT)

```
Item_Identifier,Item_Weight,Item_Type
FDA15,9.3,Dairy
DRC01,5.92,Soft Drinks
```

first line = column names

HEADER = TRUE

Item_Identifier	Item_Weight	Item_Type
FDA15	9.3	Dairy
DRC01	5.92	Soft Drinks

names are right

HEADER LEFT OUT

_c0	_c1	_c2
Item_Identifier	Item_Weight	Item_Type
FDA15	9.3	Dairy
DRC01	5.92	Soft Drinks

the header row becomes DATA

Without header=True, Spark names the columns _c0, _c1, ... and treats the first row as data.

1.5 Schema definition and printSchema()

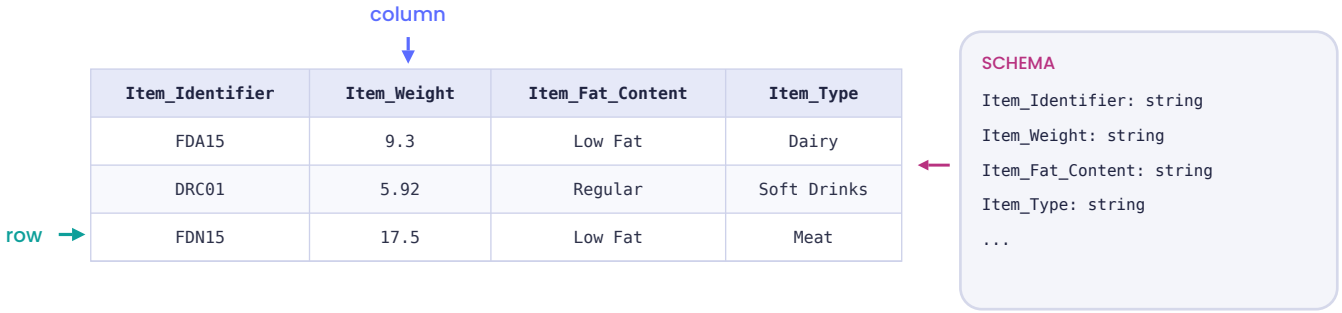
PYSPARK

```
df.printSchema()
```

DECODE THE CODE

`printSchema()`	Prints each column's name, type and whether it allows nulls - as a tree. Metadata only, so it is instant.

`printSchema()` prints the schema as a tree: each column, its type and whether it may contain nulls. It only reads metadata, so it is instant. Run it after every read and every type change.

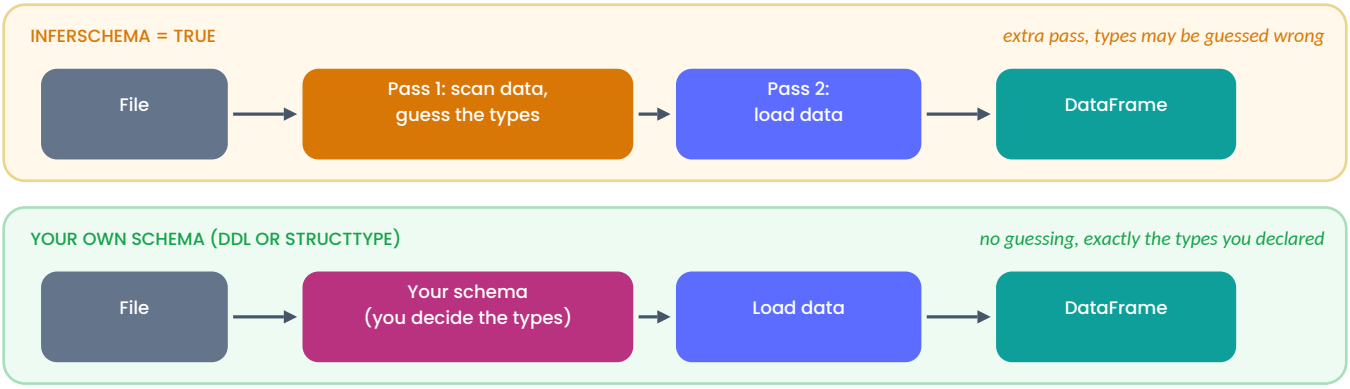


A DataFrame = named columns + a type for each column + rows spread over partitions.

TYPICAL OUTPUT WITH inferSchema=True (BigMart)

```
root
 |-- Item_Identifier: string (nullable = true)
 |-- Item_Weight: double (nullable = true)
 |-- Item_Fat_Content: string (nullable = true)
 |-- Item_Visibility: double (nullable = true)
 |-- Item_Type: string (nullable = true)
 |-- Item_MRP: double (nullable = true)
 |-- Outlet_Identifier: string (nullable = true)
 |-- Outlet_Establishment_Year: integer (nullable = true)
 |-- Outlet_Size: string (nullable = true)
 |-- Outlet_Location_Type: string (nullable = true)
 |-- Outlet_Type: string (nullable = true)
 |-- Item_Outlet_Sales: double (nullable = true)
```

inferSchema=True	Your own schema (DDL / StructType)
Needs an extra pass over the data to guess the types.	No guessing pass: Spark trusts you.
Can guess wrong (for example an ID column of digits becomes an integer).	Types are exactly what you declare.
Convenient for quick exploration.	Preferred for repeatable, production-style pipelines.



Inference is handy for exploring; a declared schema is the safer choice for repeatable pipelines.

1.6 DDL schema

DDL (Data Definition Language) is SQL's way of describing a table: `column_name TYPE`, separated by commas. Spark lets you pass exactly that, as a string, to describe your schema - the quickest way to define one.

PYSPARK

```
my_ddl_schema = '''
                    Item_Identifier STRING,
                    Item_Weight STRING,
                    Item_Fat_Content STRING,
                    Item_Visibility DOUBLE,
                    Item_Type STRING,
                    Item_MRP DOUBLE,
                    Outlet_Identifier STRING,
                    Outlet_Establishment_Year INT,
                    Outlet_Size STRING,
                    Outlet_Location_Type STRING,
                    Outlet_Type STRING,
                    Item_Outlet_Sales DOUBLE

                '''
```

DECODE THE CODE

`''' ... '''`	A triple-quoted string: lets one string span many lines (here, the DDL schema).
`STRING / DOUBLE / INT`	SQL-style type names: text, decimal number, whole number.

PYSPARK

```
df = spark.read.format('csv')\
            .schema(my_ddl_schema)\
            .option('header',True)\
            .load('/FileStore/tables/BigMart_Sales.csv')
```

PYSPARK

```
df.display()
df.printSchema()
```

OUTPUT OF printSchema() AFTER THE DDL READ

```
root
 |-- Item_Identifier: string (nullable = true)
 |-- Item_Weight: string (nullable = true)
 |-- Item_Fat_Content: string (nullable = true)
 |-- Item_Visibility: double (nullable = true)
 |-- Item_Type: string (nullable = true)
 |-- Item_MRP: double (nullable = true)
 |-- Outlet_Identifier: string (nullable = true)
 |-- Outlet_Establishment_Year: integer (nullable = true)
 |-- Outlet_Size: string (nullable = true)
 |-- Outlet_Location_Type: string (nullable = true)
 |-- Outlet_Type: string (nullable = true)
 |-- Item_Outlet_Sales: double (nullable = true)
```

- The schema is just a triple-quoted Python string; line breaks and indentation inside it do not matter.
- Column names and types go in `name TYPE` pairs separated by commas. Order must match the file's column order.
- It is passed with `.schema(my_ddl_schema)` - note that `inferSchema` is no longer needed.
- `header=True` is still needed so the first row is skipped as column names rather than read as data.

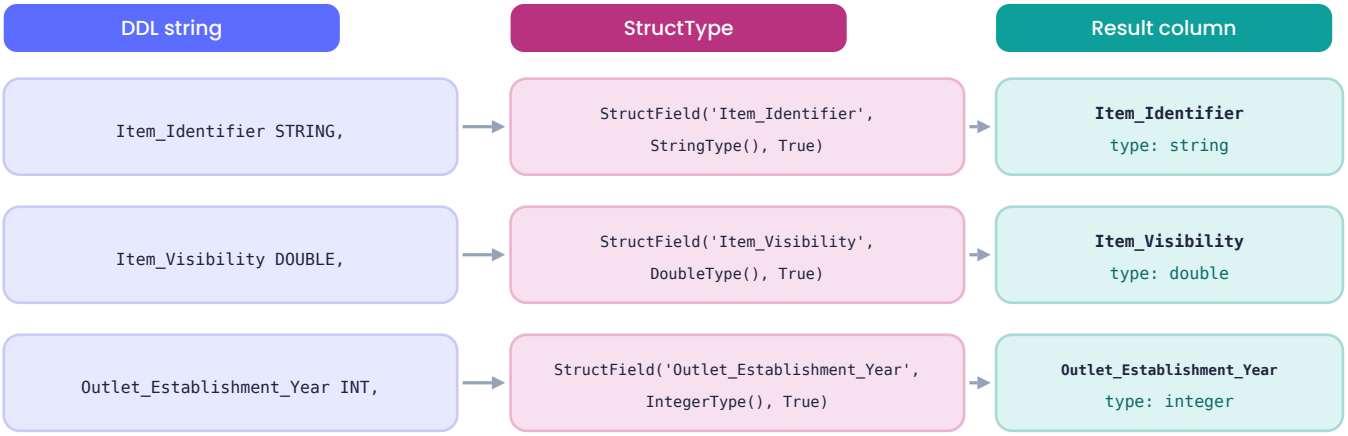
DDL type	PySpark type	Holds
`STRING`	`StringType()`	Text
`INT`	`IntegerType()`	Whole numbers (32-bit)
`DOUBLE`	`DoubleType()`	Decimal numbers
`BIGINT / FLOAT / BOOLEAN / DATE / TIMESTAMP`	`LongType / FloatType / BooleanType / DateType / TimestampType`	Extra types you will meet (not used in the notebook)

NOTEBOOK OBSERVATION

In this schema **Item_Weight is declared STRING**, whereas Spark would infer a number (double). Because of that the weight column stays text for the rest of the notebook. Spark quietly converts it to a number when you compare it with `< 10` or multiply it by Item_MRP, but **sorting** it is alphabetical (see Phase 2).

1.7 StructType schema

The second way to define a schema is programmatic: build a **StructType** (the whole table) out of **StructField**s (one per column). It is more verbose than DDL, but it is plain Python, so you can build it in code, reuse it, and nest structures.



Two spellings of the same schema. Name, type and (in StructType) nullability describe each column.

PYSPARK

```
from pyspark.sql.types import *
from pyspark.sql.functions import *
```

DECODE THE CODE

`from pyspark.sql.types import *`	Brings in all the type classes (StringType, IntegerType, DoubleType ...).
`from pyspark.sql.functions import *`	Brings in all of Spark's functions: col, lit, when, sum, avg, split ... The * means "everything".

PYSPARK · AS WRITTEN IN THE NOTEBOOK

```
my_strct_schema = StructType([
                                StructField('Item_Identifier',StringType(),True),
                                StructField('Item_Weight',StringType(),True),
                                StructField('Item_Fat_Content',StringType(),True),
                                StructField('Item_Visibility',StringType(),True),
                                StructField('Item_MRP',StringType(),True),
                                StructField('Outlet_Identifier',StringType(),True),
                                StructField('Outlet_Establishment_Year',StringType(),True),
                                StructField('Outlet_Size',StringType(),True),
                                StructField('Outlet_Location_Type',StringType(),True),
                                StructField('Outlet_Type',StringType(),True),
                                StructField('Item_Outlet_Sales',StringType(),True)
])
```

DECODE THE CODE

`StructType([...])`	A schema object: the whole table definition, built from a list of fields.
`StructField(name, type, nullable)`	One column definition: its name, a type object, and whether nulls are allowed (**True** = allowed).
`StringType() / IntegerType() / DoubleType()`	Type objects for text, whole numbers and decimal numbers.

PYSPARK

```
df = spark.read.format('csv')\
            .schema(my_strct_schema)\
            .option('header',True)\
            .load('/FileStore/tables/BigMart_Sales.csv')
```

Piece	Meaning
`StructType([...])`	The whole schema: a list of fields.
`StructField(name, type, nullable)`	One column: its name, its type object (`StringType()`, `IntegerType()` ...) and whether nulls are allowed (`True` = allowed).
`from pyspark.sql.types import *`	Brings in all the type classes (StringType, IntegerType, DoubleType ...).
`from pyspark.sql.functions import *`	Brings in all Spark functions (col, lit, when, sum, avg, split ...). Used throughout the notebook.

TWO THINGS TO KNOW ABOUT THIS PART OF THE NOTEBOOK

1. The StructType cells are Markdown cells, so they were never run. The `printSchema()` that follows therefore prints the DataFrame from the DDL read above.

2. The schema is missing a column. It lists 11 fields but the file (and the DDL schema) has 12: **Item_Type** is absent. Spark maps CSV columns by position, so if you ran it the later columns would be misaligned. It also declares every column as StringType.

A corrected, fully typed version that matches the DDL schema:

PYSPARK · CORRECTED (NOT IN THE NOTEBOOK)

```
my_strct_schema = StructType([
    StructField('Item_Identifier', StringType(), True),
    StructField('Item_Weight', StringType(), True),
    StructField('Item_Fat_Content', StringType(), True),
    StructField('Item_Visibility', DoubleType(), True),
    StructField('Item_Type', StringType(), True),
    StructField('Item_MRP', DoubleType(), True),
    StructField('Outlet_Identifier', StringType(), True),
    StructField('Outlet_Establishment_Year', IntegerType(), True),
    StructField('Outlet_Size', StringType(), True),
    StructField('Outlet_Location_Type', StringType(), True),
    StructField('Outlet_Type', StringType(), True),
    StructField('Item_Outlet_Sales', DoubleType(), True)
])
```

IMPORT * HIDES PYTHON'S BUILT-IN SUM, MIN, MAX, ROUND AND ABS

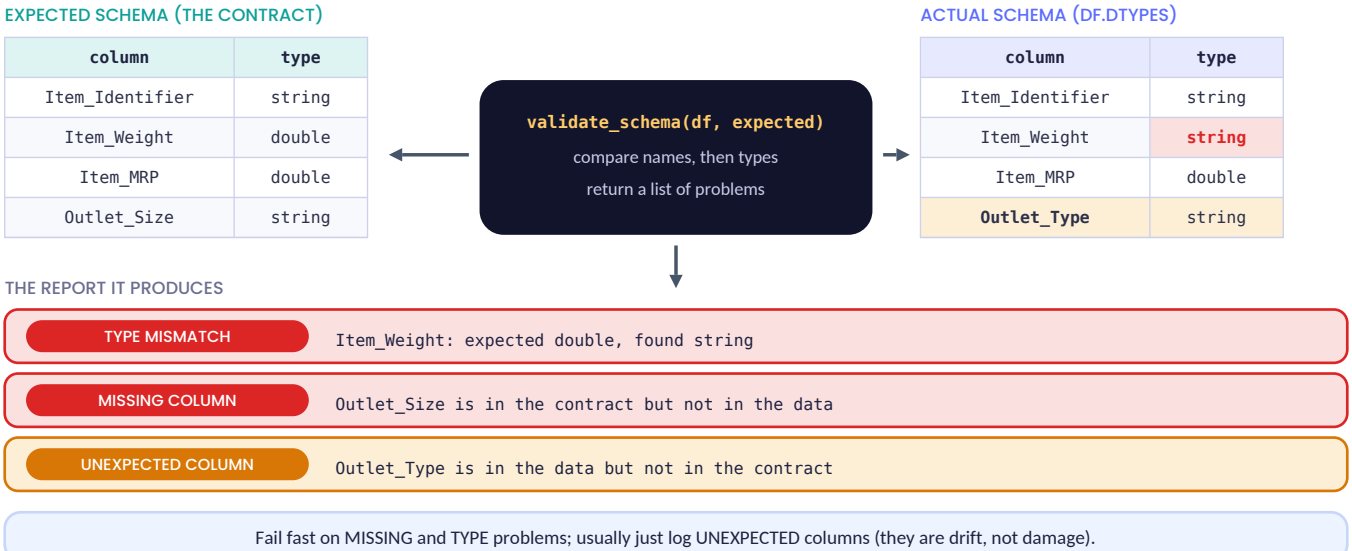
After `from pyspark.sql.functions import *`, the name `sum` means **Spark's** `sum` (which works on columns). That is what the notebook relies on in `agg(sum('Item_MRP'))`, but plain Python `sum([1, 2, 3])` will now fail. A safer habit is `from pyspark.sql import functions as F` and writing `F.sum(...)`.

	DDL string	StructType
Style	One readable SQL-like string	Python objects
Best for	Fast, flat schemas	Programmatic or nested schemas
Typos	Found only when Spark parses it	Python catches missing imports and brackets
Notebook	Executed and used	Shown, but in Markdown cells (never run)

1.8 Explicit schemas vs schema validation

BEYOND THE NOTEBOOK

Phase 1 taught you to **give Spark a schema** instead of letting it guess. That is only half of the job. An explicit schema **tells** Spark how to read the file; it does not **check** that the file still looks the way you expect. Checking is called **schema validation**.



Fail fast on MISSING and TYPE problems; usually just log UNEXPECTED columns (they are drift, not damage).

	inferSchema	Explicit schema (DDL / StructType)	Schema validation
What it does	Spark scans the data and guesses types.	You tell Spark the names and types to apply.	Your code **compares** the actual schema with the expected one.
Costs	An extra pass over the data.	Nothing extra.	Almost nothing (it only looks at metadata).
If the file changes	Types may change silently.	Values that do not fit become **null** (default mode) - silently.	You find out, and can stop or alert.
Use it for	Exploring a new file.	Repeatable reads.	Every pipeline that reads data you do not control.

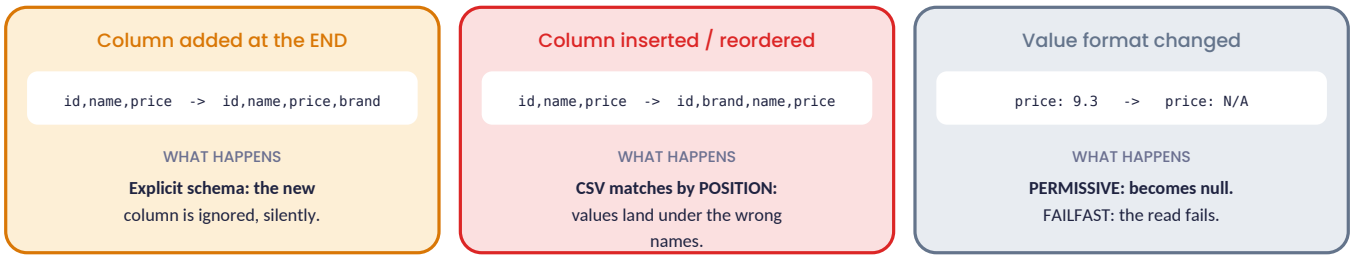
What an explicit schema does when the file drifts

Schema drift means the incoming data no longer matches what you expected: a column is added, moved, renamed, or a value changes format. These results come from reading small CSV files with the schema `'id INT, name STRING, price DOUBLE'` and `header=True`:

The file now has ...	Result with the explicit schema
a **new column at the end** (id,name,price,brand)	The new column is ignored. No error.
a column **inserted in the middle** (id,brand,name,price)	Values land under the wrong names: **name** gets the brand and **price** becomes null. No error.
a bad value (**price = N/A**)	Default mode PERMISSIVE: price becomes **null**. With FAILFAST the read fails; with DROPMALFORMED the row disappears.

WHY THE HEADER DOES NOT RESCUE YOU

With `.schema(...)` and `header=True`, Spark skips the header row and applies **your** names by **position** (the option `enforceSchema` is True by default). A file with different headers still loads under your column names. The notebook's StructType cell that omitted `Item_Type` would have shifted every later column in exactly this way.



JSON and Parquet match columns by NAME, so drift looks different: a missing column reads as null, an extra one is ignored. Either way the lesson is the same: an explicit schema TELLS Spark how to read; **only a validation step CHECKS that the file still matches.**

Choosing a read mode

PYSPARK · FAIL THE READ ON A MALFORMED ROW

```
df = spark.read.format('csv')\
          .schema(my_ddl_schema)\
          .option('header', True)\
          .option('mode', 'FAILFAST')\
          .load('/FileStore/tables/BigMart_Sales.csv')
```

DECODE THE CODE

`option('mode', ...)`	How the CSV reader treats rows that do not fit the schema: PERMISSIVE, DROPMALFORMED or FAILFAST.

mode	A row that does not fit the schema
`PERMISSIVE`	**Default.** The bad value becomes null and the row is kept.
`DROPMALFORMED`	The whole row is dropped, silently.
`FAILFAST`	The read **fails** with an error that names the record (a *Malformed records are detected in record parsing* error).

DATABRICKS RESCUED DATA COLUMN

On Databricks you can also keep values that do not fit instead of nulling them: the **rescued data column** stores them as text in an extra column so nothing is lost silently.

A reusable schema check

Validation compares **names and types**. `df.dtypes` returns a list of (column, type) pairs using Spark's short type names - `string`, `int`, `bigint`, `double`, `boolean`, `date`, `timestamp`, `decimal(10,2)`.

PYSPARK · COMPARE ACTUAL COLUMNS AND TYPES WITH THE CONTRACT

```
expected = {
    'Item_Identifier': 'string',  'Item_Weight': 'double',       'Item_Fat_Content': 'string',
    'Item_Visibility': 'double',  'Item_Type': 'string',         'Item_MRP': 'double',
    'Outlet_Identifier': 'string', 'Outlet_Establishment_Year': 'int', 'Outlet_Size': 'string',
    'Outlet_Location_Type': 'string', 'Outlet_Type': 'string',   'Item_Outlet_Sales': 'double',
}

def validate_schema(df, expected):
    actual = dict(df.dtypes)                    # {'column': 'type', ...}
    problems = []
    for name, dtype in expected.items():
        if name not in actual:
            problems.append(f'MISSING    {name}')
        elif actual[name] != dtype:
            problems.append(f'TYPE       {name}: expected {dtype}, found {actual[name]}')
    for name in actual:
        if name not in expected:
            problems.append(f'UNEXPECTED {name}')
    return problems

problems = validate_schema(df, expected)
print(problems)
```

DECODE THE CODE

`def name(args):`	Defines an ordinary Python function.
`df.dtypes`	A list of (column name, type) pairs - e.g. *('Item_MRP', 'double')*.
`f'text {x}'`	An f-string: values in braces are inserted into the text.

OUTPUT · df read with the notebook's DDL schema (Item_Weight is STRING there)

```
['TYPE       Item_Weight: expected double, found string']
```

The check immediately catches the quirk flagged back in section 1.6: the notebook's DDL declares `Item_Weight` as **STRING**, but a contract that expects **double** says it should be numeric. Now decide what to do with the problems:

PYSPARK · STOP THE PIPELINE WHEN THE CONTRACT IS BROKEN

```
if problems:
    raise ValueError('Schema check failed:\n' + '\n'.join(problems))      # fail fast
```

DECODE THE CODE

`raise`	Throws an error. Uncaught, it stops the notebook / marks the job Failed.
`.join(other, condition, how)`	Combines two DataFrames side by side by matching keys.

Problem	Usual response
MISSING column	**Fail fast.** Downstream code will break or produce wrong numbers.
TYPE mismatch	**Fail fast**, or cast on purpose and re-check (for example Item_Weight to double).
UNEXPECTED column	**Log and carry on.** It is drift to review, not damage. Accept it deliberately (schema evolution, Phase 6).

CASE SENSITIVITY

Spark matches column names case-insensitively by default, but the dictionary lookup above is case-sensitive. Lower-case both sides (`name.lower()`) if your sources vary.

INTERVIEW CORNER

Q Why define a schema instead of using inferSchema?

A inferSchema needs an extra pass over the data and can guess differently from day to day. An explicit schema is faster, repeatable and acts as a contract.

Q Does an explicit schema validate the data?

A No. It tells Spark how to read. In the default PERMISSIVE mode values that do not fit become null without an error. You need a separate validation step or `mode='FAILFAST'`.

Q What is schema drift and how would you detect it?

A The source changes shape: a column is added, removed, renamed or retyped. Compare `df.dtypes` with the expected schema on every run and alert or fail on a difference.

Q PERMISSIVE, DROPMALFORMED and FAILFAST?

A Keep the row with nulls (default) / drop the row / raise an error. FAILFAST is the safest for pipelines that must not load bad data.

Q A CSV with a schema and header=True arrives with its columns reordered. What happens?

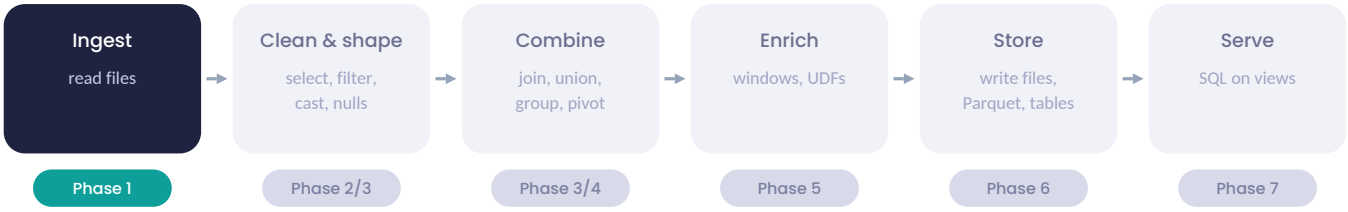
A Spark applies the schema by position, so values are loaded under the wrong names, or become null where the types do not fit - silently. Validate before trusting it.

PHASE 1 CHECKPOINT

- I can write `spark.read.format(...).option(...).load(...)` from memory.
- I know why `header` and `inferSchema` matter for CSV and that JSON does not need them.
- I can list a folder with `dbutils.fs.ls` and print a schema with `printSchema()`.
- I can define a schema both as a DDL string and as a StructType and apply it with `.schema(...)`.
- I understand that the DDL read is the DataFrame the rest of the notebook works on.
- I can explain why an explicit schema is not the same as validation.
- I know what PERMISSIVE, DROPMALFORMED and FAILFAST do and I can name three kinds of schema drift.
- I can write a function that compares `df.dtypes` with an expected schema and decide whether to fail or log.

From notebook to pipeline

Data engineers use these same skills inside **pipelines**. Here is where this phase sits, and how it is used in practice.



DATA ENGINEERING LENS

An explicit schema (DDL or StructType) acts as a **contract** with the source: the same column names and types every run, instead of whatever Spark infers that day.

inferSchema costs an extra pass over the data and can guess wrongly, so it suits exploration more than repeatable jobs.

Run **printSchema()** straight after every read - the notebook does, and it catches type surprises before they travel downstream.

Use **dbutils.fs.ls** to confirm a landing file exists and to copy its exact path before reading.

Beyond the notebook: an explicit schema does not **validate**. In PERMISSIVE mode bad values silently become null, so compare **df.dtypes** with an expected schema on every run, or read with mode FAILFAST.

Glossary

Concepts from this phase, then every **code term** the decoder explained, in the order you met them.

Concept	In plain words
Schema	The list of column names and their data types.
Schema inference	Spark scanning the data to guess the types (inferSchema=True).
DDL schema	A schema written as a SQL-style string: 'name TYPE, name TYPE'.
StructType / StructField	A schema built from Python objects: the table, and each column.
Nullable	Whether a column is allowed to contain nulls.
JSON Lines	A JSON file with one complete record per line (multiLine=False).
Reader API	format -> option(s) -> (schema) -> load.
Schema validation	Checking that the actual columns and types match the expected ones.
Schema drift	The source's shape changes: a column is added, removed, renamed or retyped.
PERMISSIVE / DROPMALFORMED / FAILFAST	Keep bad values as null / drop the row / raise an error.

Code terms decoded in this phase

Code term	Meaning
`( ... )`	Wrapping a chain in parentheses lets it run over several lines without backslashes.
`spark.read`	Starts a **DataFrameReader**: the object you configure and then use to load a file.
`.format('csv')`	Chooses the file format: **csv**, **json**, **parquet** ... (the same call is used when writing).
`.option(key, value)`	Sets **one** setting for the reader or writer. Call it again for each extra setting.
`header`	**True** = the first row of the CSV holds column names, not data.
`.schema(my_schema)`	Hands Spark the schema to use instead of guessing it.
`.load(path)`	Points Spark at the file or folder and returns a DataFrame. (Lazy until an action needs the data.)
`spark`	The SparkSession - your entry point to Spark. Databricks creates it for you, already named **spark**.
`inferSchema`	**True** = Spark scans the data to guess each column's type. Without it, every CSV column is a string.
`\ (backslash)`	Line continuation: the statement carries on in the next line. Nothing may follow it on the same line.
`multiLine`	**False** = one JSON record per line. **True** = a record may span several lines.
`/FileStore/tables/`	The Databricks folder (DBFS) where uploaded files live.
`.display()`	Databricks' table viewer. It is an **action**, so it makes Spark run the job.
`dbutils.fs.ls(path)`	Databricks utility that **lists** the files in a storage folder, like *ls* in a terminal.
`printSchema()`	Prints each column's name, type and whether it allows nulls - as a tree. Metadata only, so it is instant.
`''' ... '''`	A triple-quoted string: lets one string span many lines (here, the DDL schema).
`STRING / DOUBLE / INT`	SQL-style type names: text, decimal number, whole number.
`from pyspark.sql.types import *`	Brings in all the type classes (StringType, IntegerType, DoubleType ...).
`from pyspark.sql.functions import *`	Brings in all of Spark's functions: col, lit, when, sum, avg, split ... The * means "everything".
`StructType([...])`	A schema object: the whole table definition, built from a list of fields.
`StructField(name, type, nullable)`	One column definition: its name, a type object, and whether nulls are allowed (**True** = allowed).
`StringType() / IntegerType() / DoubleType()`	Type objects for text, whole numbers and decimal numbers.
`option('mode', ...)`	How the CSV reader treats rows that do not fit the schema: PERMISSIVE, DROPMALFORMED or FAILFAST.
`def name(args):`	Defines an ordinary Python function.
`df.dtypes`	A list of (column name, type) pairs - e.g. *('Item_MRP', 'double')*.
`f'text {x}'`	An f-string: values in braces are inserted into the text.

Code term	Meaning
`raise`	Throws an error. Uncaught, it stops the notebook / marks the job Failed.
`.join(other, condition, how)`	Combines two DataFrames side by side by matching keys.

Check yourself

Try each question first, then click **Show answer**. Every answer has a **back** link that returns you to the question.

Questions

Q1 Which option makes the first row of a CSV the column names? Show answer

Q2 If you give neither `inferSchema` nor a schema, what type do CSV columns get? Show answer

Q3 What is the difference between `multiLine=False` and `multiLine=True` for JSON? Show answer

Q4 Why would the notebook's StructType schema not work if you ran it? Show answer

Q5 When would you prefer a StructType over a DDL string? Show answer

Q6 An explicit schema is in place. A column is inserted in the middle of the file. What happens? Show answer

Q7 Write the check that finds columns whose type differs from an expected dictionary. Show answer

Hands-on tasks

Task 1 Read the BigMart CSV with the DDL schema and print the schema. Show solution

Task 2 Write a StructType schema for three columns: Item_Identifier (text), Item_MRP (decimal) and Outlet_Establishment_Year (whole number). Show solution

Task 3 Compare the BigMart DataFrame's types with the expected ones and raise an error if a column is missing. Show solution

Answers and solutions

A1 `option('header', True)`. Without it the columns are named `_c0, _c1 ...` and the header row becomes data. back to question

A2 Every column is read as **string**. back to question

A3 False: each line of the file is one complete record (JSON Lines). True: a record may be spread over several lines, as in a pretty-printed file. back to question

A4 The cells are Markdown, so they never ran; it lists 11 fields while the file has 12 (`Item_Type` is missing, so columns shift); and every field is StringType. back to question

A5 When the schema is built in Python code, reused, or nested. A DDL string is quicker for flat schemas. back to question

A6 Spark applies the schema by **position**: values land under the wrong names, and values that do not fit become null - with no error. Validate names and types, or use FAILFAST. back to question

A7 `[n for n, t in expected.items() if dict(df.dtypes).get(n) != t]` - then fail or log. back to question

Task solutions

Solution 1 back to task

SOLUTION

```
df = spark.read.format('csv')\
            .schema(my_ddl_schema)\
            .option('header',True)\
            .load('/FileStore/tables/BigMart_Sales.csv')

df.printSchema()
```

Solution 2 back to task

SOLUTION

```
from pyspark.sql.types import *

my_schema = StructType([
    StructField('Item_Identifier', StringType(), True),
    StructField('Item_MRP', DoubleType(), True),
    StructField('Outlet_Establishment_Year', IntegerType(), True)
])
```

Solution 3 back to task

SOLUTION

```
problems = validate_schema(df, expected)      # function from section 1.8
if any(p.startswith('MISSING') for p in problems):
    raise ValueError('\n'.join(problems))
```

PREVIOUS
← Phase 0 · Spark Foundations

NEXT
Phase 2 · Beginner Transformations →

Tip: keep all nine PDFs in the same folder so the previous / next links open the right file.